



Facultad de Ciencias de la vida y tecnologías

Asignatura:

Aplicaciones Web II 6to “B”

Docente:

Ing. John Cevallos Macias.

Tema: LigaFit

Autores:

Anchundia Pillasagua Karen Yelena

Cedeño Gomez Johantony Jose

Moreira Garcia Marly Sinay

Fecha:

10 de junio de 2026



HITO 2 – IMPLEMENTACIÓN DE MÓDULOS DEL SISTEMA LIGAFIT

1. Introducción

En este segundo hito se desarrollaron los módulos principales del sistema LigaFit utilizando el lenguaje Go y el framework Chi Router. El objetivo fue implementar la lógica de negocio, almacenamiento en memoria, rutas API REST y operaciones CRUD para cada uno de los módulos asignados al equipo.

El proyecto se desarrolló siguiendo una arquitectura basada en Models, Storage, Services, Handlers y Routes, permitiendo una estructura organizada y escalable para futuras mejoras.

2. Módulo 1 – Gestión de Deportistas

2.1. Objetivo

Permitir el registro y administración de los deportistas pertenecientes a la liga deportiva.

Funcionalidades implementadas

- Registrar deportistas.
- Consultar todos los deportistas.
- Consultar deportistas por ID.
- Actualizar información de deportistas.
- Eliminar deportistas.
- Asignación automática de categoría según la edad.

Validaciones implementadas

- Nombre obligatorio.
- Nombre con mínimo 3 caracteres.
- Edad mayor a cero.
- Género obligatorio.
- Teléfono obligatorio.
- Teléfono con mínimo 10 dígitos.
- Deporte obligatorio.

Archivos desarrollados

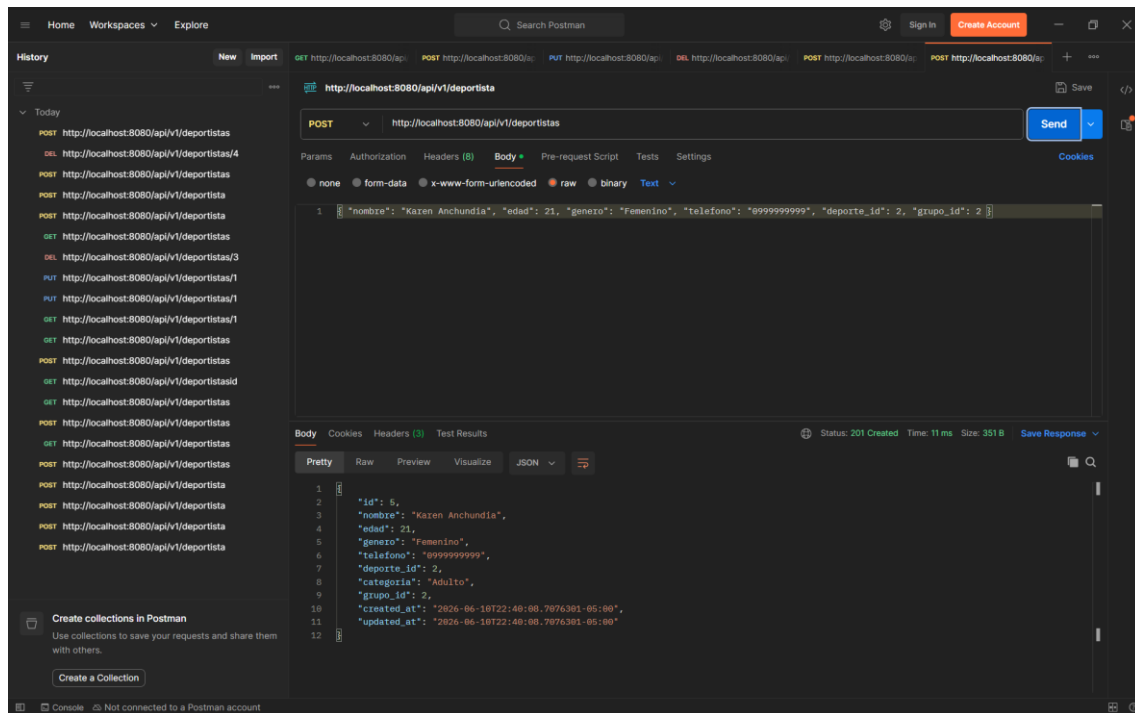
- deportista.go
- deportista_memory.go
- deportista_service.go
- deportista_handler.go
- routes.go



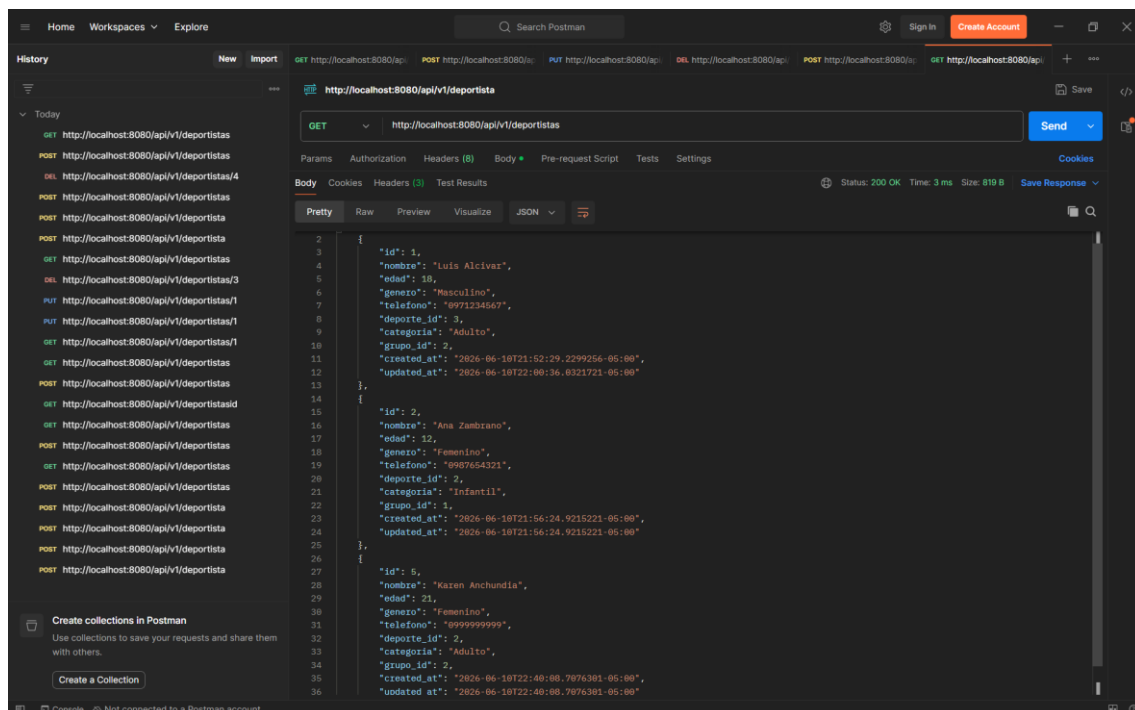
Evidencia

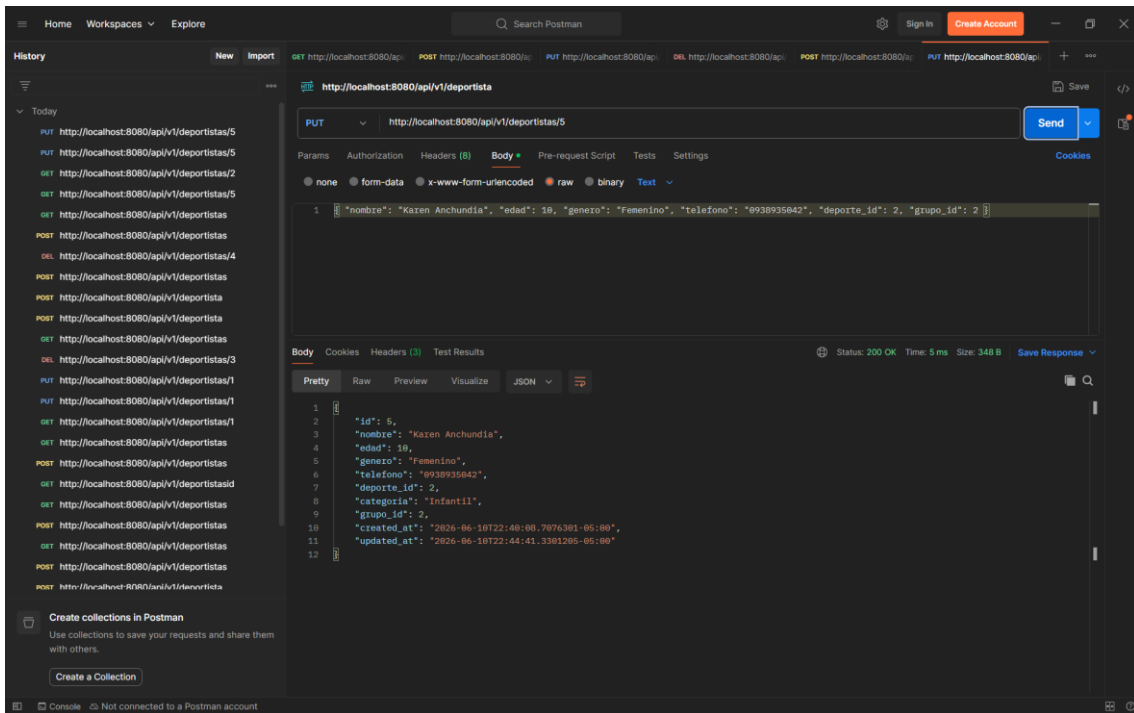
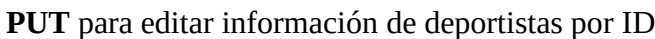
Se realizaron pruebas exitosas mediante Postman utilizando los métodos:

POST para agregar deportista



GET para listar deportistas y mostrar deportistas por ID







DELETE para eliminar deportistas por ID

History: Today
DELETE http://localhost:8080/api/v1/deportistas/5
PUT http://localhost:8080/api/v1/deportistas/5
PUT http://localhost:8080/api/v1/deportistas/5
GET http://localhost:8080/api/v1/deportistas/2
GET http://localhost:8080/api/v1/deportistas/5
GET http://localhost:8080/api/v1/deportistas
POST http://localhost:8080/api/v1/deportistas
DELETE http://localhost:8080/api/v1/deportistas/4
POST http://localhost:8080/api/v1/deportistas
POST http://localhost:8080/api/v1/deportista
POST http://localhost:8080/api/v1/deportista
DELETE http://localhost:8080/api/v1/deportistas
DELETE http://localhost:8080/api/v1/deportistas/3
PUT http://localhost:8080/api/v1/deportistas/1
PUT http://localhost:8080/api/v1/deportistas/1
GET http://localhost:8080/api/v1/deportistas/1
GET http://localhost:8080/api/v1/deportistas
POST http://localhost:8080/api/v1/deportistas
GET http://localhost:8080/api/v1/deportistasid
GET http://localhost:8080/api/v1/deportistas
POST http://localhost:8080/api/v1/deportistas
GET http://localhost:8080/api/v1/deportistas
POST http://localhost:8080/api/v1/deportistas
POST http://localhost:8080/api/v1/deportistas

Create collections in Postman
Use collections to save your requests and share them with others.
Create a Collection

DELETE http://localhost:8080/api/v1/deportistas/5
Status: 204 No Content Time: 3 ms Size: 64 B Save Response

History: Today
GET http://localhost:8080/api/v1/deportistas
DELETE http://localhost:8080/api/v1/deportistas/5
PUT http://localhost:8080/api/v1/deportistas/5
PUT http://localhost:8080/api/v1/deportistas/5
GET http://localhost:8080/api/v1/deportistas/2
GET http://localhost:8080/api/v1/deportistas/5
GET http://localhost:8080/api/v1/deportistas
POST http://localhost:8080/api/v1/deportistas
DELETE http://localhost:8080/api/v1/deportistas/4
POST http://localhost:8080/api/v1/deportistas
POST http://localhost:8080/api/v1/deportista
DELETE http://localhost:8080/api/v1/deportistas
DELETE http://localhost:8080/api/v1/deportistas/3
PUT http://localhost:8080/api/v1/deportistas/1
PUT http://localhost:8080/api/v1/deportistas/1
GET http://localhost:8080/api/v1/deportistas/1
GET http://localhost:8080/api/v1/deportistas
POST http://localhost:8080/api/v1/deportistas
GET http://localhost:8080/api/v1/deportistasid
GET http://localhost:8080/api/v1/deportistas
POST http://localhost:8080/api/v1/deportistas
GET http://localhost:8080/api/v1/deportistas
POST http://localhost:8080/api/v1/deportistas
POST http://localhost:8080/api/v1/deportistas

Create collections in Postman
Use collections to save your requests and share them with others.
Create a Collection

GET http://localhost:8080/api/v1/deportistas
Status: 200 OK Time: 3 ms Size: 582 B Save Response



3. Módulo 2 – Seguimiento Físico y Nutricional

3.1. Objetivo

Permitir registrar información física de los deportistas para monitorear su evolución y apoyar el seguimiento nutricional.

Funcionalidades implementadas

- Registrar seguimientos físicos.
- Consultar seguimientos registrados.
- Consultar seguimiento por ID.
- Actualizar información física.
- Eliminar registros de seguimiento.
- Gestión de recomendaciones nutricionales.

Validaciones implementadas

- Peso mayor a cero.
- Altura mayor a cero.
- Altura entre 0.5 y 2.5 metros.
- Validación de datos obligatorios.

Archivos desarrollados

- seguimiento_fisico.go
- nutricionista.go
- recomendacion_nutricional.go
- seguimiento_memory.go
- recomendacion_memory.go
- seguimiento_service.go
- seguimiento_fisico_handler.go
- routes.go

Evidencia

Se realizaron pruebas CRUD exitosas mediante Bruno verificando la correcta creación, consulta, actualización y eliminación de registros.

POST



My Workspace

Overview Environments POST Untitled +

POST http://localhost:8080/api/v1/seguimientos

Params Body Headers Auth JSON Prettify Response Headers

```
1 {
2   "deportista_id": 1,
3   "entrenador_id": 1,
4   "peso": 60.5,
5   "altura": 1.75
6 }
```

```
1 {
2   "id": 2,
3   "deportista_id": 1,
4   "entrenador_id": 1,
5   "peso": 60.5,
6   "altura": 1.75,
7   "imc": 19.76,
8   "estado_fisico": "Normal",
9   "requiere_evaluacion_nutricional": false,
10  "fecha_registro": "2026-06-10",
11  "created_at": "2026-06-10T22:22:54.5733756-05:00",
12  "updated_at": "2026-06-10T22:22:54.5733756-05:00"
13 }
```

201 Created 8ms 276B

API Specs +

Try Bruno Ultimate

Buscar

25°C Prac. despejado

22:22 10/6/2026

GET

My Workspace

Overview Environments GET Untitled +

GET http://localhost:8080/api/v1/seguimientos

Params Body Headers Auth JSON Prettify Response Headers Timeline

```
1 {
2   "deportista_id": 1,
3   "entrenador_id": 1,
4   "peso": 60.5,
5   "altura": 1.75
6 }
```

```
1 [
2   {
3     "id": 2,
4     "deportista_id": 1,
5     "entrenador_id": 1,
6     "peso": 60.5,
7     "altura": 1.75,
8     "imc": 19.76,
9     "estado_fisico": "Normal",
10    "requiere_evaluacion_nutricional": false,
11    "fecha_registro": "2026-06-10",
12    "created_at": "2026-06-10T22:22:54.5733756-05:00",
13    "updated_at": "2026-06-10T22:22:54.5733756-05:00"
14   }
15 ]
```

200 OK 8ms 276B

API Specs +

Try Bruno Ultimate

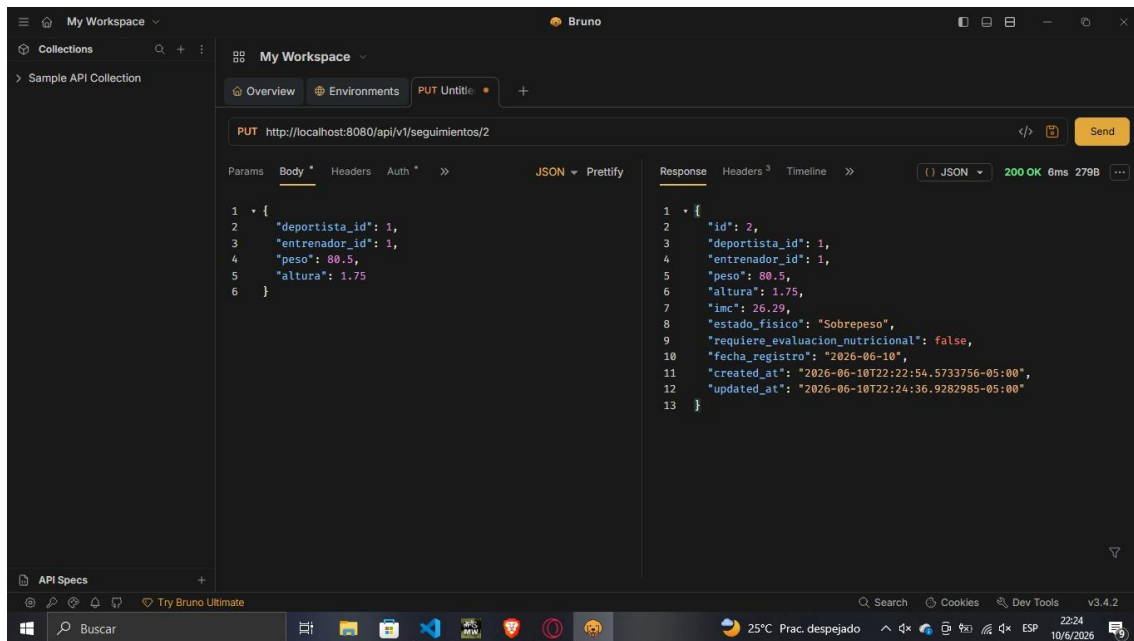
Buscar

25°C Prac. despejado

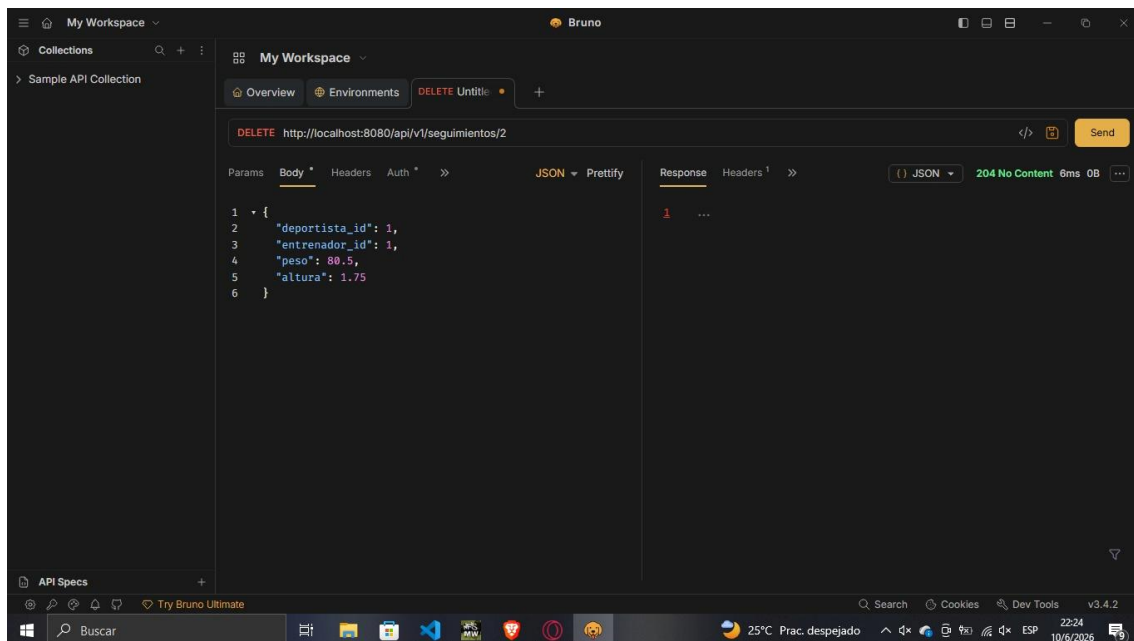
22:23 10/6/2026



PUT



DELETE



4. Módulo 3 – Competencias y Participaciones

4.1. Objetivo

Administrar competencias deportivas y controlar la participación de los deportistas en cada evento.

Funcionalidades implementadas

- Registrar competencias.
- Consultar competencias.



- Actualizar competencias.
- Eliminar competencias.
- Registrar participaciones.
- Consultar participaciones.
- Actualizar participaciones.
- Eliminar participaciones.

Validaciones implementadas

- Nombre obligatorio.
- Lugar obligatorio.
- Categoría obligatoria.
- Validación de datos requeridos para participación.

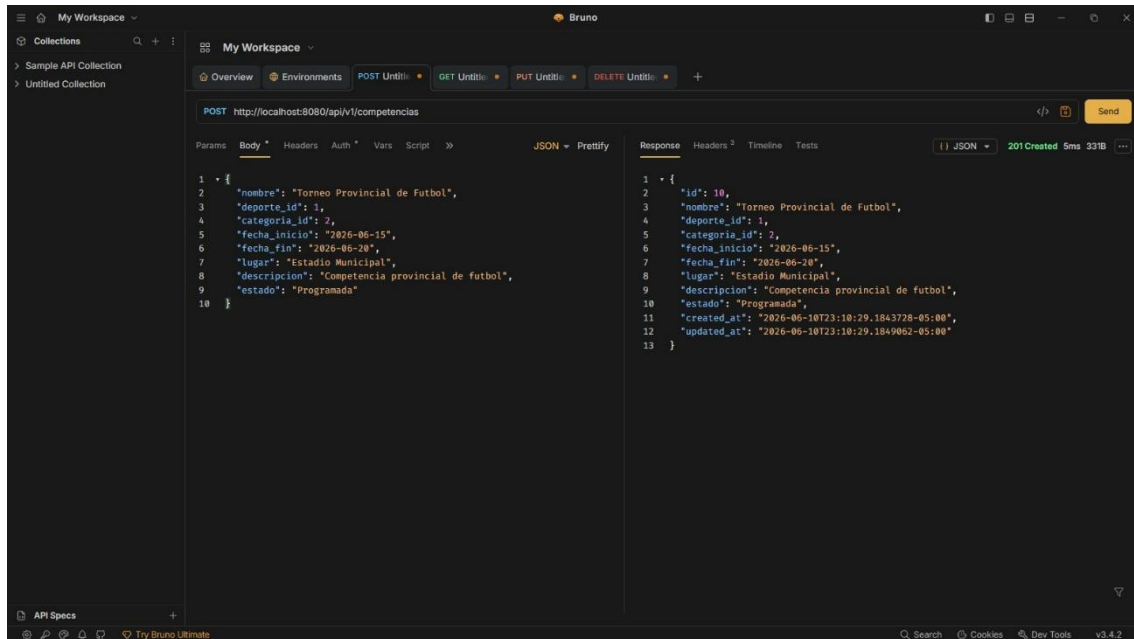
Archivos desarrollados

- competencia.go
- participacion.go
- competencia_memory.go
- competencia_service.go
- participacion_service.go
- competencia_handler.go
- participacion_handler.go
- routes.go

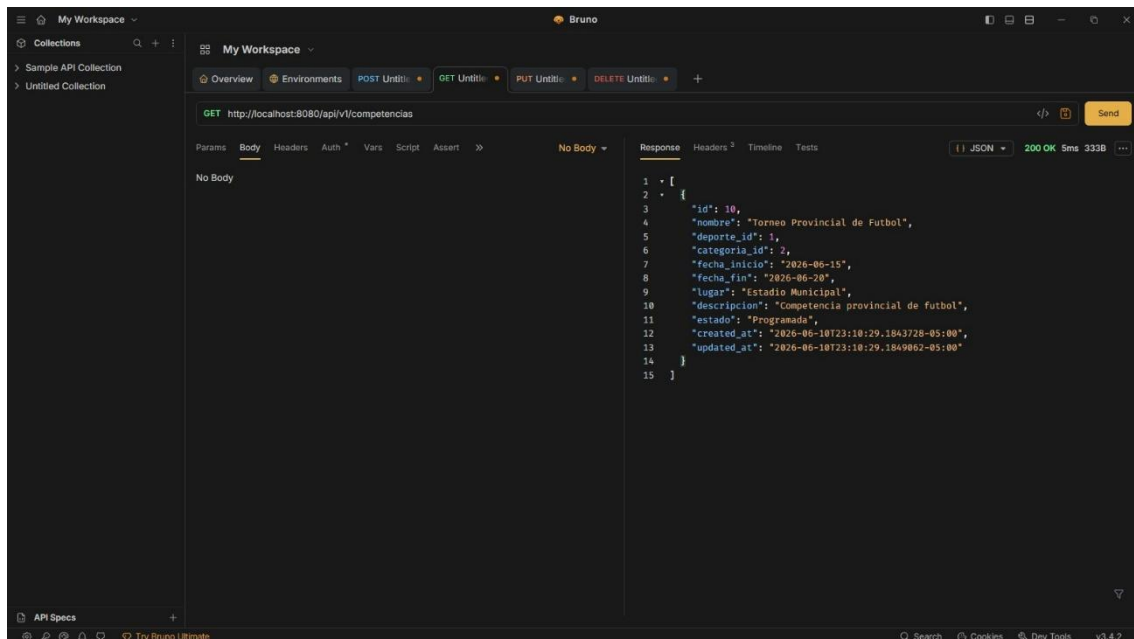


Evidencia

Se realizaron pruebas completas mediante Bruno verificando el funcionamiento correcto de todas las operaciones CRUD.

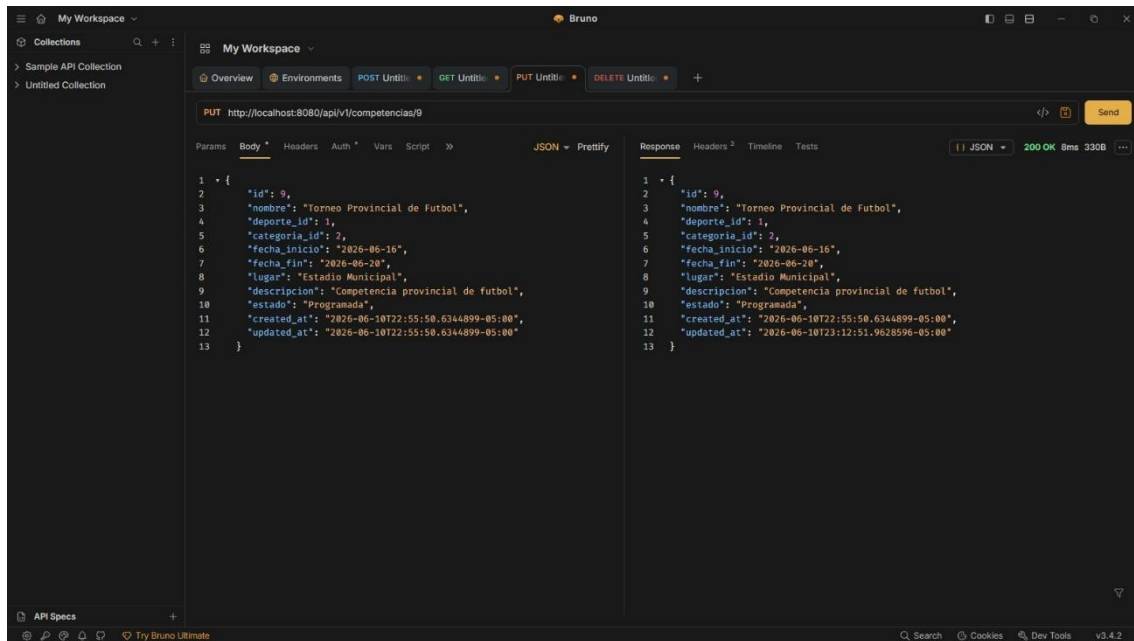


GET

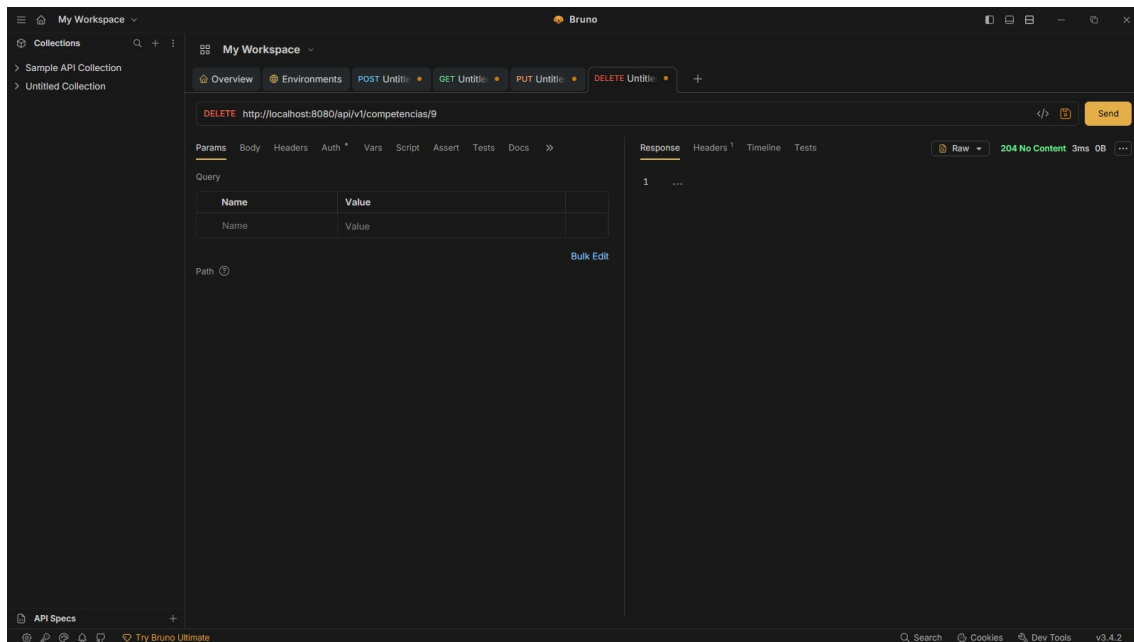




PUT



DELETE



5. Control de versiones con Git y GitHub

Durante el desarrollo se utilizó Git para el control de versiones y GitHub como repositorio remoto.

Se trabajó mediante ramas independientes:

- feature/modulo-deportistas
- feature/modulo-seguimiento
- feature/modulo-competencias



Cada integrante desarrolló su módulo de forma independiente realizando commits progresivos que evidencian la evolución del proyecto.

6. Resultados obtenidos

- Implementación de los tres módulos principales.
- Desarrollo completo de operaciones CRUD.
- Integración de rutas REST.
- Validaciones de negocio.
- Almacenamiento temporal en memoria.
- Uso de Git y GitHub para control de versiones.
- Pruebas funcionales exitosas mediante Bruno y Postman.

7. Conclusiones

La arquitectura basada en Models, Storage, Services y Handlers permitió organizar correctamente el proyecto.

El uso de Git facilitó el trabajo colaborativo mediante ramas independientes.

Las pruebas realizadas demostraron el correcto funcionamiento de los módulos implementados.

El sistema quedó preparado para futuras integraciones con bases de datos persistentes y nuevas funcionalidades.